PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 04 -

AUTOMAÇÃO COM IAC E CI/CD (TERRAFORM + GITHUB ACTIONS)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 13
O QUE VOCÊ VIU NESTA AULA? 14
REFERÊNCIAS 15

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Automatizar é primordial em times modernos de infraestrutura. Se até aqui você aprendeu a escrever código de infraestrutura modular e flexível, nesta aula vamos naturalmente dar o próximo passo: **integrar o Terraform com pipelines de CI/CD**. A ideia é simples e poderosa: garantir que cada alteração de infraestrutura seja versionada, validada e aplicada de forma previsível, sem depender de execuções manuais na máquina do desenvolvedor ou do time de Infraestrutura.

Vamos aprender como configurar o Terraform em conjunto com o **GitHub Actions**, construindo pipelines que cuidam desde a validação do código, passando pelo **terraform plan**, até a aplicação nos ambientes. Mais do que apenas rodar comandos, vamos falar sobre **boas práticas de automação**, como uso de backends remotos, controle de credenciais e workflows seguros. Ao final da aula, você verá como transformar um repositório Git em uma esteira completa de entrega de infraestrutura como código.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

Chegou o momento de explorar como times podem automatizar o ciclo de vida da infraestrutura. No Hands On**,** veremos como transformar um repositório GitHub criando uma pipeline que valida, planeja e aplica mudanças de infraestrutura com o Terraform. A proposta é que toda modificação de código passe por uma sequência controlada de etapas antes de se refletir em recursos reais no ambiente, garantindo consistência e previsibilidade.

Ao longo da aula, será configurado um **workflow no GitHub Actions** que cuida de tarefas essenciais: instalação do Terraform, inicialização do projeto, validação da sintaxe, execução do ***terraform plan*** e provisionamento em ambientes distintos. Essa abordagem permite que diferentes membros do time colaborem com segurança, sabendo que qualquer alteração será avaliada e testada antes de ser efetivada.

Essa prática mostra como a IaC ganha força ao ser integrada a **pipelines de CI/CD**, trazendo governança e reduzindo riscos em ambientes críticos. Mais do que apenas aplicar mudanças, o time passa a ter um processo de entrega contínua para infraestrutura, alinhado ao que já se faz no desenvolvimento de software, mas agora estendido também ao mundo da cloud.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

Manter a infraestrutura com base apenas em execuções manuais pode até funcionar em ambientes pequenos, mas rapidamente se torna insustentável em times de médio e grande porte. Cada engenheiro aplicando comandos em sua máquina introduz riscos: inconsistências de versões, configurações diferentes e falta de rastreabilidade.

É aqui que entram as **pipelines de CI/CD**. Elas centralizam a execução, eliminam as execuções manuais e criam um fluxo previsível: cada alteração é validada, testada e aplicada de acordo com regras definidas pelo time. Os benefícios são:

- **Governança**: toda mudança fica registrada, auditada e é rastreável.
- **Consistência**: o mesmo código gera os mesmos resultados em qualquer ambiente.
- **Velocidade**: times aplicam mudanças mais rápido sem abrir mão da segurança.
- **Confiança**: alterações críticas passam por revisão e validação antes de afetar a produção.

Quando usar cada fluxo

Para esclarecer sua utilização, os bullets a seguir sintetizam pontos que te ajudarão a escolher devidamente cada fluxo.

Fluxo manual:

- Aprender e explorar o Terraform.
- Realizar revisões detalhadas de planos de infraestrutura em ambientes de produção.
- Testar mudanças antes de implementá-las em ambientes de desenvolvimento ou pré-produção.

Automatizado:

- Ambientes de desenvolvimento e testes.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

- Implantação contínua de infraestrutura.
- Projetos que exigem escalabilidade e alta frequência de atualizações.

Configurando GitHub Actions para Terraform

O **GitHub Actions** é uma das ferramentas mais acessíveis para times que já versionam sua infraestrutura no GitHub. Ele permite definir **workflows** em arquivos YAML que são disparados por eventos (push, PR, merge).

Estrutura de workflows

Um workflow mínimo para Terraform pode conter três etapas principais:

```
name: Terraform CI/CD

on:
  pull_request:
    branches: [ "main" ]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: hashicorp/setup-terraform@v2
      - run: terraform init
      - run: terraform validate
      - run: terraform plan -var-file="dev.tfvars"
```

Esse fluxo é inicializado a cada **pull request** para a branch principal. Ele garante que o código seja inicializado e validado antes de aplicar.

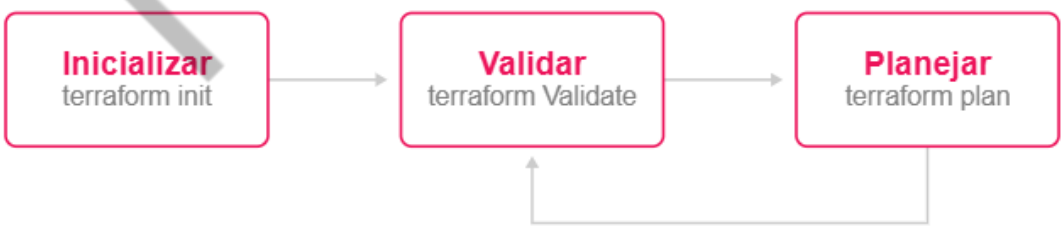


Figura 1 – Fluxo
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Lint, plan e apply automatizados

Além do ***validate*** e ***plan***, é comum incluir etapas de ***lint*** para padronização de código, usando ferramentas como tflint. O ***apply*** pode ser automatizado para ambientes de desenvolvimento, mas geralmente em produção exige mecanismos extras de aprovação.

```
      - name: Lint Terraform
        uses: terraform-linters/setup-tflint@v1
      - run: tflint --recursive
```

Controle de ambientes com branches

Para aplicar as configurações, podemos separar por diretórios e usar branches ***feature/*** para aplicar no ambiente em questão. Após o provisionamento ser concluído, faz-se o merge para a main.

Porém, uma estratégia frequente é mapear branches para ambientes. Por exemplo:

- Branch ***develop*** → aplica em ambiente **dev**.
- Branch ***main*** → aplica em ambiente **prod**.

```
      on:
        push:
          branches:
            - "develop"
            - "main"

      jobs:
        terraform:
          runs-on: ubuntu-latest
          steps:
            - uses: actions/checkout@v3
            - uses: hashicorp/setup-terraform@v2
            - run: terraform init
            - run: terraform plan -var-file="${{ github.ref_name
}}.tfvars"
            - run: terraform apply -auto-approve -var-file="${{
github.ref_name }}.tfvars"
```

Perceba que neste caso, o arquivo .tfvars selecionado recebe o valor da Branch através da variável ${{ github.ref_name }}.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Casos de uso avançados

À medida que os times amadurecem, as pipelines de Terraform evoluem para incluir **aprovações manuais, rollback e auditoria**.

Workflows com aprovação manual

Em ambientes críticos, não é recomendável aplicar automaticamente. O GitHub Actions oferece o recurso de Environments com approval gates, onde um responsável deve aprovar a execução antes do ***apply***.

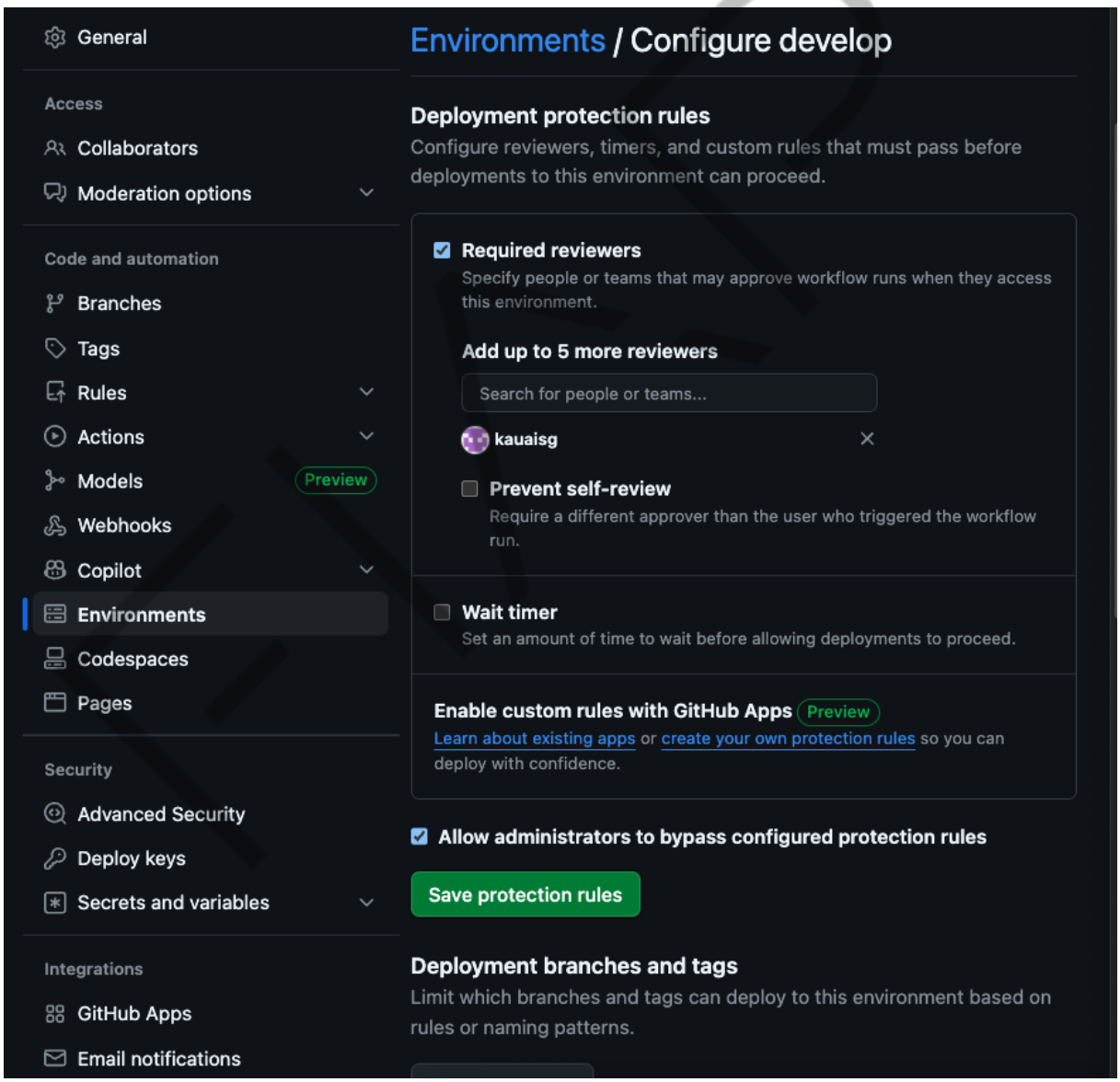


Figura 2 – Tela de configuração
Fonte: GitHub Actions (2025)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Estratégias de rollback

O Terraform não possui um comando “rollback” nativo. No entanto, existem estratégias como:

- Manter **planos versionados** e aplicar o último plano válido em caso de erro.
- Usar **tags de repositório** para marcar versões estáveis da infraestrutura.
- Integrar com **workflows de destruição seletiva** ou restaurar state remoto.

Veja nos exemplos essas estratégias aplicadas ao Workflow do GitHub Actions.

Rollback via tags de versão

Cada vez que o time aplica uma mudança estável em produção, cria-se uma **tag** no repositório (ex.: `infra-v1.0`). Se algo falhar em `infra-v1.1`, basta reverter para a tag anterior e rodar o apply novamente.

```
name: Rollback Infra by Tag

on:
  workflow_dispatch:
    inputs:
      version:
        description: 'Tag de versão para rollback'
        required: true

jobs:
  rollback:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          ref: ${{ github.event.inputs.version }}

      - uses: hashicorp/setup-terraform@v2
      - run: terraform init
      -   run:   terraform   apply   -auto-approve   -var-
file="prod.tfvars"
```

Esse fluxo permite escolher manualmente uma tag de infraestrutura anterior.

Rollback usando planos salvos

Em pipelines maduras, o `terraform plan` pode ser salvo como arquivo `.tfplan` versionado. Se a aplicação falhar, é possível reexecutar o plano anterior.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
name: Rollback with Saved Plan

on:
  workflow_dispatch:

jobs:
  rollback:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: hashicorp/setup-terraform@v2
      - run: terraform init

      # Reaplica o último plano aprovado
      - run: terraform apply -auto-approve last-
successful.tfplan
```

Aqui, o desafio é gerenciar o armazenamento desses planos (ex.: S3, GCS ou artefatos do próprio GitHub Actions).

Rollback com branch estável

Uma prática comum é manter uma branch ***production-stable.*** O pipeline de produção sempre aplica essa branch. Se algo der errado em ***main***, basta reverter o merge e a ***production-stable*** volta a apontar para a última versão válida.

```
name: Rollback Infra Stable Branch

on:
  push:
    branches: [ "production-stable" ]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: hashicorp/setup-terraform@v2
      - run: terraform init
      - run: terraform apply -auto-approve -var-
file="prod.tfvars"
```

O rollback aqui é simples: basta forçar a branch ***production-stable*** a apontar para o commit anterior estável.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Auditoria de mudanças

Outro benefício das pipelines é a **auditoria centralizada**. Cada execução do workflow gera logs completos, associando quem fez a mudança, qual foi o ***plan*** aprovado e quando ocorreu o ***apply***. Isso ajuda tanto em **compliance** quanto em **investigações de incidentes**.

Ferramentas como Atlantis também podem ser integradas ao GitHub para trazer comentários automáticos de *plan* e ***apply*** dentro dos pull requests, tornando o processo mais transparente para o time.

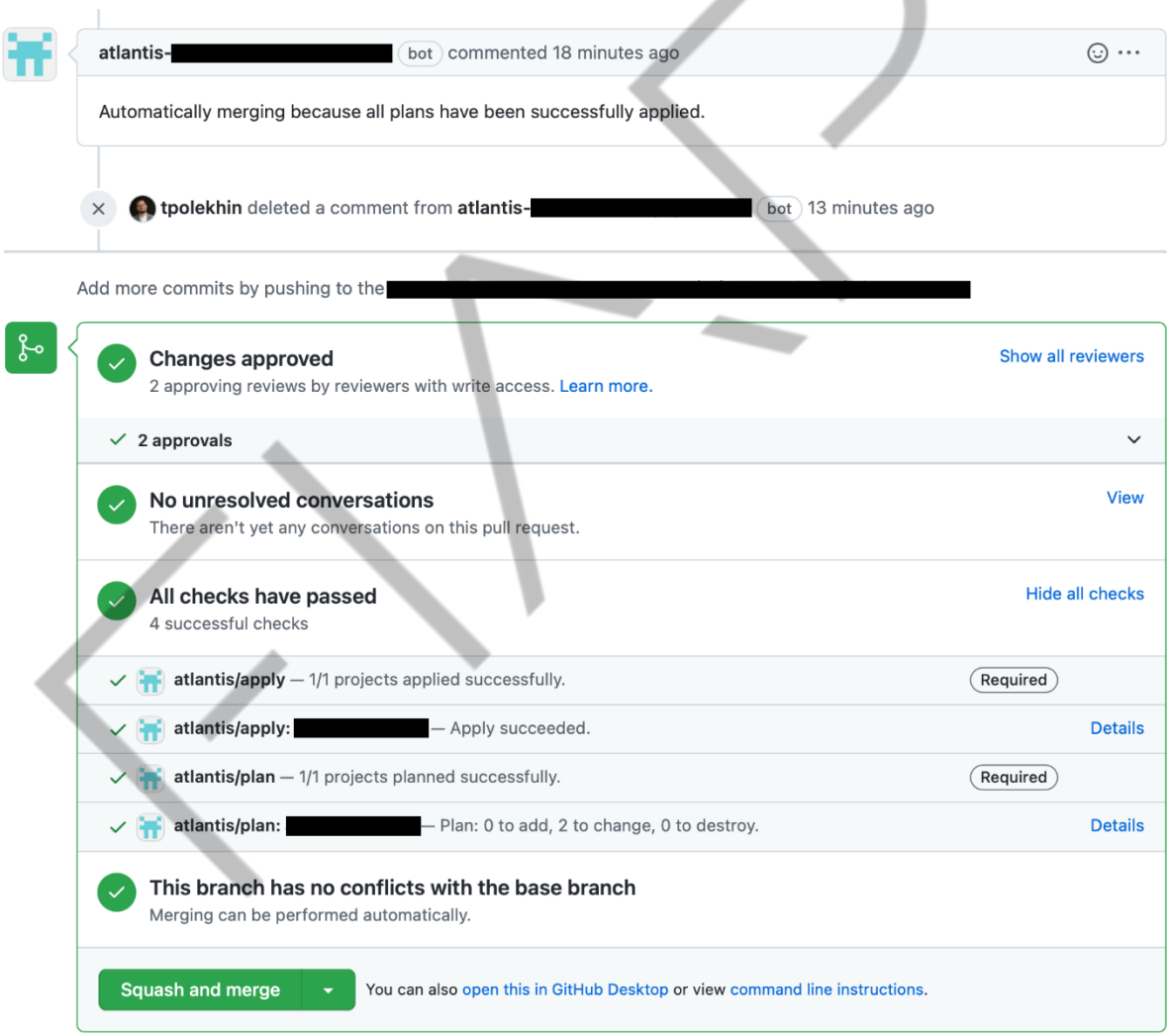


Figura 3 – tela de pull request
Fonte: Atlantis (2025)

Você pode conhecer um pouco mais sobre o Atlantis na DevOpsSchool.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Conclusão

Automatizar infraestrutura com **CI/CD** muda a dinâmica do time: deixa de ser um processo manual e arriscado para se tornar um **fluxo contínuo, auditável e previsível**. Começa simples, com validação e plan em cada PR, e evolui para pipelines sofisticados com lint, aprovação manual, rollback e auditoria completa.

Essa é a direção do mercado: **infraestrutura entregue como software**, com qualidade e governança de ponta a ponta.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

À medida que os times crescem e os ambientes ficam mais complexos, a necessidade de **padronização, governança e reuso** em fluxos de automação se torna crítica. É aqui que ferramentas como o GitHub Actions se destacam, não apenas pela facilidade de integração com repositórios, mas pela capacidade de criar fluxos robustos e reutilizáveis em escala organizacional.

Um artigo interessante vem do próprio GitHub. Nele, a empresa mostra como construiu **padrões internos de pipelines**, garantindo que times diferentes dentro da organização pudessem adotar automação de forma consistente e segura. Isso inclui desde a criação de **workflows reutilizáveis** até a definição de **políticas de governança** que controlam como, quando e por quem os pipelines podem ser executados.

Esse exemplo reforça uma tendência crescente: pipelines não são mais configurados isoladamente por cada time, mas sim tratados como **ativos compartilhados**, que seguem diretrizes globais da organização. Isso acelera a adoção de automação, reduz falhas e cria um ecossistema onde IaC e CI/CD são aplicados com eficiência e segurança em larga escala.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como a automação transforma a prática de **Infraestrutura como Código** em um processo previsível, auditável e seguro. Discutimos porque fluxos manuais se tornam rapidamente insustentáveis em times maiores e como pipelines de **CI/CD com GitHub Actions** oferecem governança e consistência no ciclo de vida da infraestrutura.

Vimos na prática a estrutura de **workflows para Terraform**, incluindo etapas de validação, lint, plan e apply, além de estratégias para separar ambientes de desenvolvimento e produção. Também conhecemos casos de uso avançados, como a implementação de aprovações manuais, rollback baseado em versões estáveis e auditoria centralizada das mudanças.

Com isso, ficou claro que a automação não é apenas uma conveniência, mas sim um **pilar estratégico** para times que querem escalar a infraestrutura com segurança e eficiência, mantendo alinhamento com as práticas mais modernas de DevOps.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

ATLANTIS. **Using Atlantis (Guide)**. 2025. Disponível em: https://www.runatlantis.io/docs/using-atlantis. Acesso em: 21 out. 2025.

GITHUB. **Reuse Workflows (Reusable Workflows)**. 2025. Disponível em: https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows. Acesso em: 21 out. 2025.

GITHUB. **Reviewing Deployments (Environment Approvals).** 2025. Disponível em: https://docs.github.com/actions/managing-workflow-runs/reviewing-deployments. Acesso em: 21 out. 2025.

GITHUB. **Using Environments for Deployment (Approvals & Secrets)**. 2025. Disponível em: https://docs.github.com/actions/deployment/targeting-different-environments/using-environments-for-deployment. Acesso em: 21 out. 2025.

HASHICORP. **Backend Type: S3 (Remote State).** 2025. Disponível em: https://developer.hashicorp.com/terraform/language/backend/s3. Acesso em: 21 out. 2025.

HASHICORP. **Setup Terraform (GitHub Actions).** GitHub Marketplace, 2025. Disponível em: https://github.com/marketplace/actions/hashicorp-setup-terraform. Acesso em: 21 out. 2025.

TERRAFORM-LINTERS. **TFLint – A Pluggable Terraform Linter**. 2025. Disponível em: https://github.com/terraform-linters/tflint . Acesso em: 21 out. 2025.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

terraform. github. Ci/cd, automação.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com